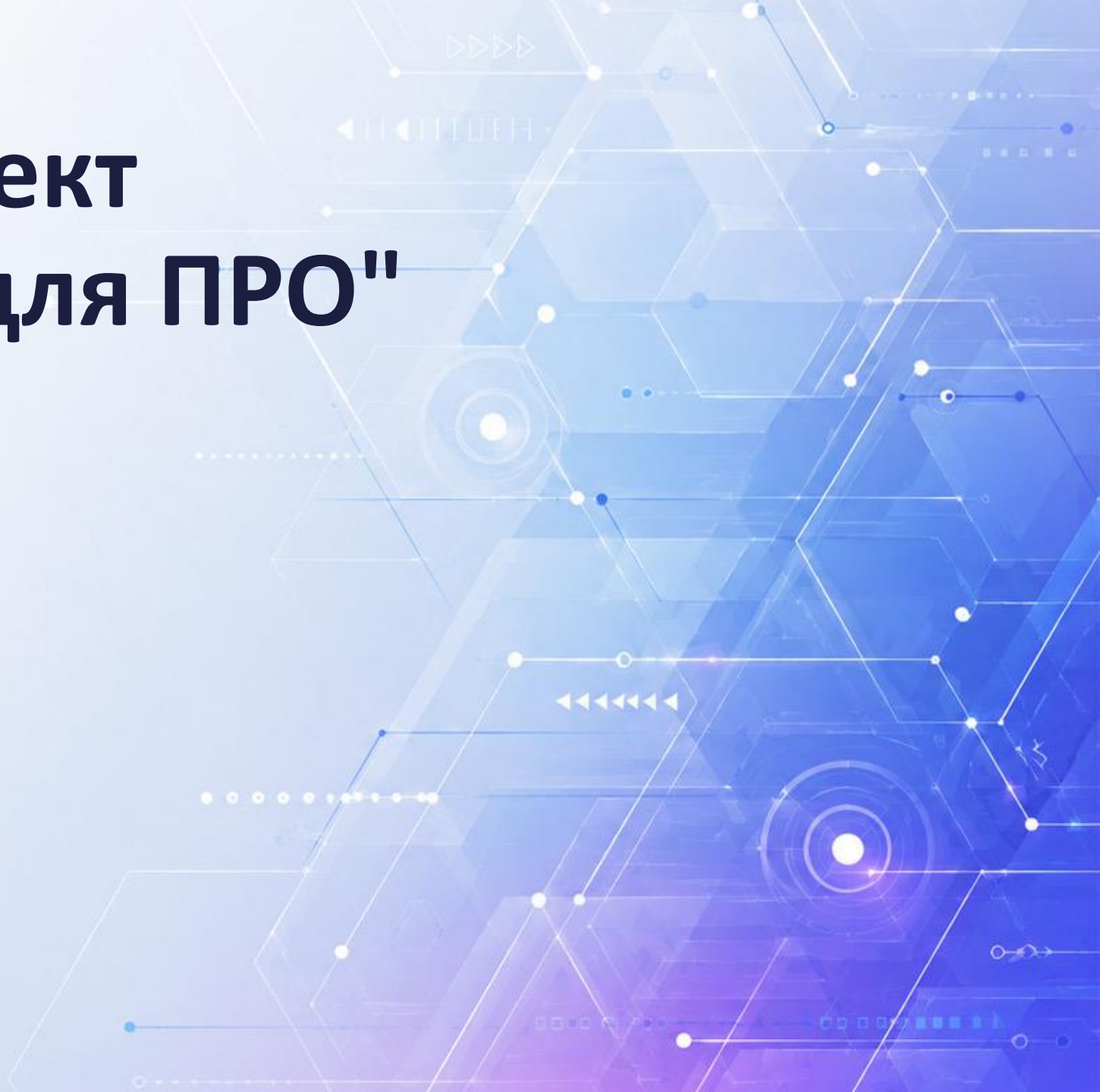


Дипломный проект "Дата-инженер для ПРО"

Рудских Александр



Анализ данных

Первичный анализ данных по продажам произведен при помощи Pandas. Изучена структура данных, выявлено что в данных нет дубликатов в первичном ключе Invoice ID. Выявлены столбцы для слоя NDS, данные из которых станут основой для таблиц-справочников:

- Уникальные города и филиалы → `nds.locations`
- Уникальные типы клиентов → `nds.customer_types`
- Уникальные линейки продуктов → `nds.products`
- Уникальные способы оплаты → `nds.payment_types`
- Таблица фактов продажи → `nds.sales`

Анализ данных

```
In [6]: 1 len(df['Invoice ID'].unique())
```

```
Out[6]: 1000
```

Invoice ID уникален во всем датасете, может быть использован как первичный ключ для таблицы продаж.

```
In [7]: 1 for i in df.columns.to_list():  
2     print(i, end = ' - количество уникальных значений ----- ')  
3     print(len(df[i].unique()))
```

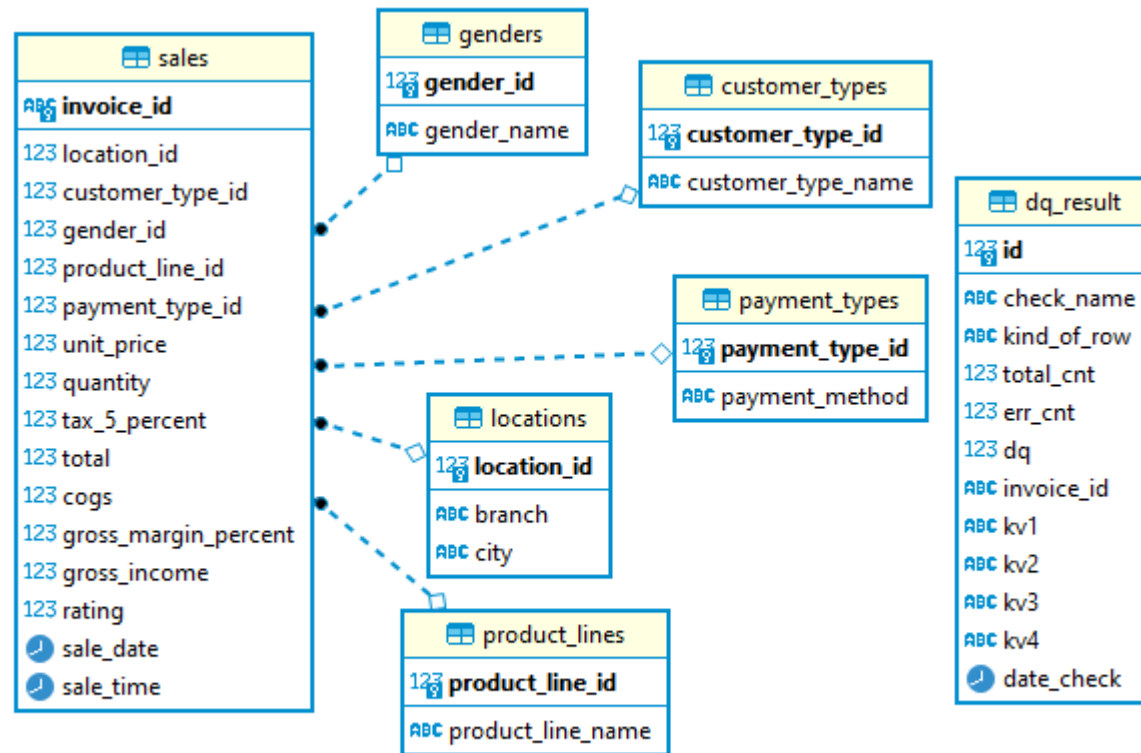
```
Invoice ID - количество уникальных значений ----- 1000  
Branch - количество уникальных значений ----- 3  
City - количество уникальных значений ----- 3  
Customer type - количество уникальных значений ----- 2  
Gender - количество уникальных значений ----- 2  
Product line - количество уникальных значений ----- 6  
Unit price - количество уникальных значений ----- 943  
Quantity - количество уникальных значений ----- 10  
Tax 5% - количество уникальных значений ----- 990  
Total - количество уникальных значений ----- 990  
Date - количество уникальных значений ----- 89  
Time - количество уникальных значений ----- 506  
Payment - количество уникальных значений ----- 3  
cogs - количество уникальных значений ----- 990  
gross margin percentage - количество уникальных значений ----- 1  
gross income - количество уникальных значений ----- 990  
Rating - количество уникальных значений ----- 61
```

Создание БД и схем (PostgreSQL)

- На стороне PostgreSQL создана База данных `final_work_de`, схема `nds`, таблицы справочников и таблица фактов `nds.sales`.
- Создана схема `dds` и таблица фактов `dds.sales_fact`.
- Также создана таблица для размещения результатов запуска проверок качества данных `nds.dq_result`.
- создан пользователь `python_user`, которому предоставлены права к созданным объектам.

Создание БД и схем (PostgreSQL)

- NDS



- DDS

sales_fact
fact_id
ABC invoice_id
ABC branch
ABC city
ABC customer_type
ABC gender
ABC product_line
ABC payment_method
123 unit_price
123 quantity
123 tax_amount
123 total_amount
123 cogs
123 gross_income
123 rating
🕒 sale_date
🕒 sale_time
123 year
123 month
123 quarter
ABC day_of_week

Заполнение таблиц слоя NDS

```
In [5]: 1 engine = create_engine('postgresql://python_user:python_pass@localhost:5432/final_work_de')
```

```
In [6]: 1 #функция для загрузки данных на слой NDS, нормализованный факт и справочники
2 def load_nds(df, engine):
3
4     locations = df[['Branch', 'City']].rename(columns={'Branch': 'branch', 'City': 'city'}).drop_duplicates()
5     locations.to_sql('locations', engine, schema='nds', if_exists='append', index=False)
6
7     customer_types = df[['Customer type']].rename(columns={'Customer type': 'customer_type_name'}).drop_duplicates()
8     customer_types.to_sql('customer_types', engine, schema='nds', if_exists='append', index=False)
9
10    genders = df[['Gender']].rename(columns={'Gender': 'gender_name'}).drop_duplicates()
11    genders.to_sql('genders', engine, schema='nds', if_exists='append', index=False)
12
13    product_lines = df[['Product line']].rename(columns={'Product line': 'product_line_name'}).drop_duplicates()
14    product_lines.to_sql('product_lines', engine, schema='nds', if_exists='append', index=False)
15
16    payment_types = df[['Payment']].rename(columns={'Payment': 'payment_method'}).drop_duplicates()
17    payment_types.to_sql('payment_types', engine, schema='nds', if_exists='append', index=False)
18
19    # маппинги для внешних ключей для join
20    locations_map = pd.read_sql("SELECT location_id, branch, city FROM nds.locations", engine)
21    customer_types_map = pd.read_sql("SELECT customer_type_id, customer_type_name FROM nds.customer_types", engine)
22    genders_map = pd.read_sql("SELECT gender_id, gender_name FROM nds.genders", engine)
23    product_lines_map = pd.read_sql("SELECT product_line_id, product_line_name FROM nds.product_lines", engine)
24    payment_types_map = pd.read_sql("SELECT payment_type_id, payment_method FROM nds.payment_types", engine)
25
26    # Преобразуем DataFrame фактов
27    facts = df.copy()
28
```

Заполнение таблиц слоя DDS

Запуск скрипта для заполнения таблицы выполним из файла nds_to_dds.sql

```
1 # Открытие файла и чтение его содержимого
2 with open("nds_to_dds.sql") as file:
3     sql_query = file.read()
4
5 # Подключение к базе данных и выполнение запроса
6 with engine.connect() as con:
7     con.execute(sql_query)
```

```
INSERT INTO dds.sales_fact (
    invoice_id, branch, city, customer_type, gender, product_line, payment_method,
    unit_price, quantity, tax_amount, total_amount, cogs, gross_income, rating,
    sale_date, sale_time, year, month, quarter, day_of_week
)
SELECT
    s.invoice_id,
    l.branch,
    l.city,
    ct.customer_type_name AS customer_type,
    g.gender_name AS gender,
    pl.product_line_name AS product_line,
    pt.payment_method,
    s.unit_price,
    s.quantity,
    s.tax_5_percent AS tax_amount,
    s.total AS total_amount,
    s.cogs,
    s.gross_income,
    s.rating,
    s.sale_date,
    s.sale_time,
    EXTRACT(YEAR FROM s.sale_date)::INTEGER AS year,
    EXTRACT(MONTH FROM s.sale_date)::INTEGER AS month,
    EXTRACT(QUARTER FROM s.sale_date)::INTEGER AS quarter,
    TO_CHAR(s.sale_date, 'Day') AS day_of_week
FROM nds.sales s
JOIN nds.locations l ON s.location_id = l.location_id
JOIN nds.customer_types ct ON s.customer_type_id = ct.customer_type_id
JOIN nds.genders g ON s.gender_id = g.gender_id
JOIN nds.product_lines pl ON s.product_line_id = pl.product_line_id
JOIN nds.payment_types pt ON s.payment_type_id = pt.payment_type_id
.
```

Проверка качества данных

- Проверка качества данных осуществлена с помощью готового скрипта с набором проверок data quality check.sql. Каждая проверка по результатам запуска создает 2 типа строк:

T - содержит информацию об общем количестве проверенных строк, количестве ошибочных строк и уровне качества данных, который определяется следующим образом - $1 - (\text{кол-во проверенных строк}) / (\text{кол-во ошибочных})$. Получаем долю верных данных.

R - сами ошибочные строки.

- Для строк R основной атрибут это `invoice_id`, также зарезервированы поля для дополнительных данных `kv1`, `kv2`, `kv3` и `kv4`, в которые можно размещать дополнительные атрибуты.
- В каждой строке заполняется поле `date_check` с указанием момента запуска проверки.

Проверка качества данных

```
--dq
--DQ_1
--Отрицательные значения в основных сущностях
INSERT INTO nds.dq_result (
    check_name, kind_of_row, total_cnt, err_cnt, dq, invoice_id, kv1, kv2, kv3, kv4, date_check
)
with t1 as
(select *,
case when unit_price < 0
or quantity < 0
or tax_5_percent < 0
or total < 0 then 1 else 0 end as err_flg
from nds.sales
)
select
'DQ_1' as check_name,
'T' as kind_of_row,
count(err_flg) as total_cnt,
sum(err_flg) as err_cnt,
1.0-sum(err_flg)*1.0/count(err_flg) as dq,
null as invoice_id,
null as kv1,
null as kv2,
null as kv3,
null as kv4,
now() as date_check
from t1
union all
select
'DQ_1' as check_name,
'R' as kind_of_row,
null as total_cnt,
null as err_cnt,
null as dq,
invoice_id,
unit_price::text as kv1,
quantity::text as kv2,
tax_5_percent::text as kv3,
total::text as kv4,
now() as date_check
from t1
where err_flg = 1
;
```

Проверка качества данных

Для примера изменена проверка DQ_2 таким образом, чтоб она давала срабатывания. Условно она считает, что рейтинг больше 9 является не корректным.

dq_result 1 X													
select * from nds.dq_result Введите SQL выражение чтобы отфильтровать результаты													
	123 id	ABC check_name	ABC kind_of_row	123 total_cnt	123 err_cnt	123 dq	ABC invoice_id	ABC kv1	ABC kv2	ABC kv3	ABC kv4	date_check	
4	12	DQ_1	T	1 000	0	1	[NULL]	[NULL]	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
5	13	DQ_2	T	1 000	151	0,85	[NULL]	[NULL]	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
6	14	DQ_2	R	[NULL]	[NULL]	[NULL]	750-67-8428	9.1	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
7	15	DQ_2	R	[NULL]	[NULL]	[NULL]	226-31-3081	9.6	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
8	16	DQ_2	R	[NULL]	[NULL]	[NULL]	636-48-8204	9.9	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
9	17	DQ_2	R	[NULL]	[NULL]	[NULL]	145-94-9061	9.6	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
10	18	DQ_2	R	[NULL]	[NULL]	[NULL]	132-32-9879	9.5	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
11	19	DQ_2	R	[NULL]	[NULL]	[NULL]	326-78-5178	9.5	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
12	20	DQ_2	R	[NULL]	[NULL]	[NULL]	120-06-4233	9.3	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
13	21	DQ_2	R	[NULL]	[NULL]	[NULL]	285-68-5083	10.0	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
14	22	DQ_2	R	[NULL]	[NULL]	[NULL]	347-34-2234	10.0	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
15	23	DQ_2	R	[NULL]	[NULL]	[NULL]	109-28-2512	9.9	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	
16	24	DQ_2	R	[NULL]	[NULL]	[NULL]	393-65-2792	9.6	[NULL]	[NULL]	[NULL]	2026-06-09 20:55:56.633	

Save Cancel Script 200 468 Rows: 1 468 строк получено - 3ms (2ms полнч.). июн. 09. 20:57:25

Dashboard (Power BI)



Вывод

В рамках выполнения работы была проведена комплексная работа с данными:

- данными были изучены
- подготовлены к загрузке в базу данных (создание таблиц фактов и справочников)
- сформированы проверки качества данных
- данные были перемещены на слой для формирования отчетности
- подготовлен дашборд для визуализации данных

СПАСИБО